Research Assignment 01

Classification models, ensemble methods, activation functions, and core decision-tree concepts

Course	**Hope to Skills — Free Artificial Intelligence Advance Course**
Student	Abdul Rehman
Prepared for	Irfan Malik Sir and Dr. Sheraz Sir

Academic submission prepared in a structured report format

Document field	Details
Research assignment	Research Assignment 01
Student name	Abdul Rehman
Course	Hope to Skills — Free Artificial Intelligence Advance Course
Instructors	Irfan Malik and Dr. Sheraz
Objective	To explain important classification models, ensemble methods, activation functions, tree-splitting criteria, bootstrap sampling, and evaluation metrics.

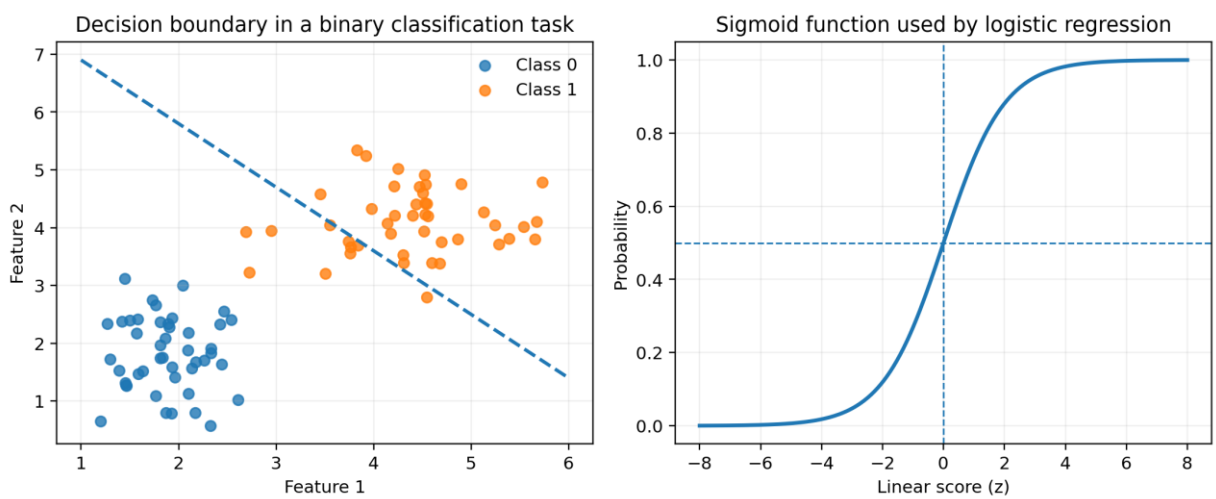
Report overview

This submission is written in a structured academic style and answers every question in the assignment directly.
I used concise explanations first, then added practical reasoning, advantages, limitations, and application context so the report shows both conceptual understanding and applied thinking.

Question 1 — Logistic Regression as a Classification Model

Logistic Regression is a supervised learning algorithm used for classification, especially binary classification problems. Unlike linear regression, which predicts a continuous number, logistic regression predicts the probability that an observation belongs to a class. It first computes a linear score from the features and then passes that score through the sigmoid function, which maps the output to a value between 0 and 1.

Mathematically, the model estimates p = 1 / (1 + e^-z), where z = b0 + b1x1 + ... + bnxn. A threshold such as 0.5 can then be used to decide the final class label. Because the output is probabilistic, logistic regression is easy to interpret and is often used when transparency matters.



Advantages: It is fast, interpretable, and works well when the relationship between features and log-odds is approximately linear. It is also a strong baseline model for many binary classification tasks.

Limitations: It may underperform when the class boundary is highly non-linear or when there are complex feature interactions that are not explicitly engineered.

Real-world application: A practical use case is predicting whether a hospital patient is at risk of being readmitted within 30 days. Variables such as age, diagnosis history, prior admissions, length of stay, and medication patterns can be used as inputs. The model outputs a probability of readmission, which can help healthcare teams decide which patients need extra follow-up.

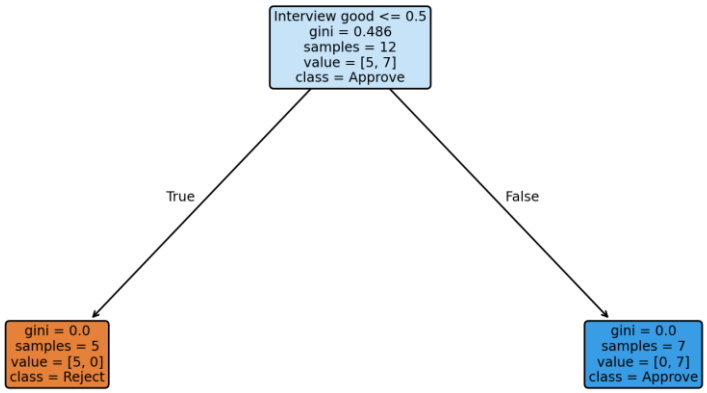
Interpretation: In this example, logistic regression is valuable because the hospital does not only need a yes/no prediction; it also needs a risk score that can be explained to clinicians.

Question 2 — Decision Tree Classification Model

A Decision Tree is a supervised learning model that classifies data by repeatedly splitting it into smaller and more homogeneous groups. Each internal node represents a test on a feature, each branch represents an outcome of that test, and each leaf node gives the final prediction.

The model chooses the best split at each step by using measures such as Gini impurity or entropy. The main goal is to create child nodes that are more pure, meaning they contain examples from one class as much as possible.

Simple decision tree structure for an admission decision example



Advantages: Decision trees are easy to understand, easy to visualize, and can handle both numerical and categorical data. They also capture non-linear decision boundaries without requiring feature scaling.

Limitations: A single tree can overfit the training data, especially if it is allowed to grow very deep. It can also be unstable, meaning a small change in the data may produce a different tree.

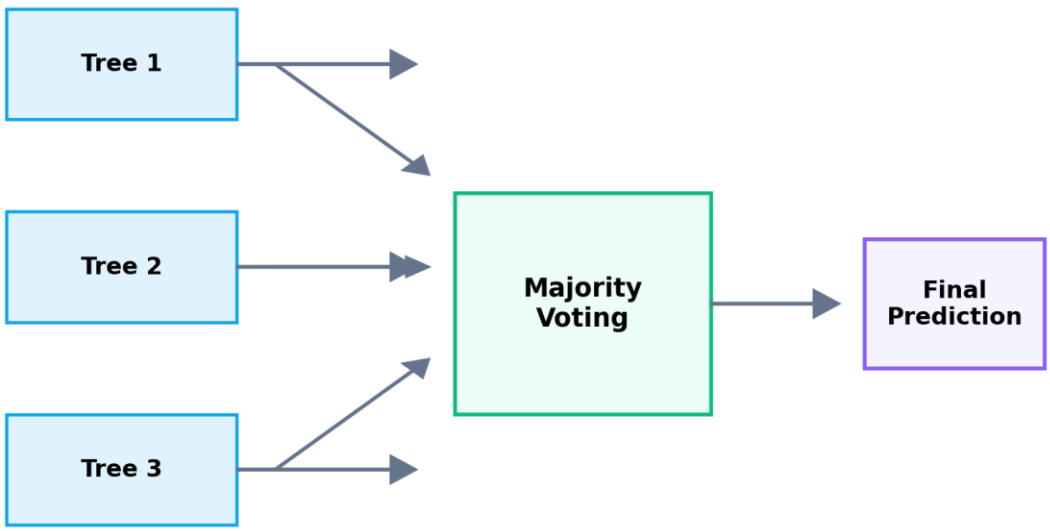
Real-world application: A useful example is university admission screening. A decision tree can consider test score, previous academic percentage, interview performance, and prerequisite completion, then classify a candidate as approved or not approved. The model is useful here because each split can be understood as a clear decision rule.

Question 3 — Random Forest Classification Model

Random Forest is an ensemble classification model built from a large collection of decision trees. Instead of relying on one tree, it trains many trees using different bootstrap samples of the data and random subsets of features. During prediction, the individual trees vote, and the majority vote becomes the final class prediction.

This approach improves upon a single decision tree in two major ways. First, it reduces variance, which means the model is less likely to overfit. Second, random feature selection makes the trees less correlated with one another, which improves the quality of the ensemble.

Random forest combines many decision trees built from different bootstrap samples



Advantages: Random forest is robust, accurate, relatively easy to train, and handles noisy data better than a single tree. It also provides useful information such as feature importance.

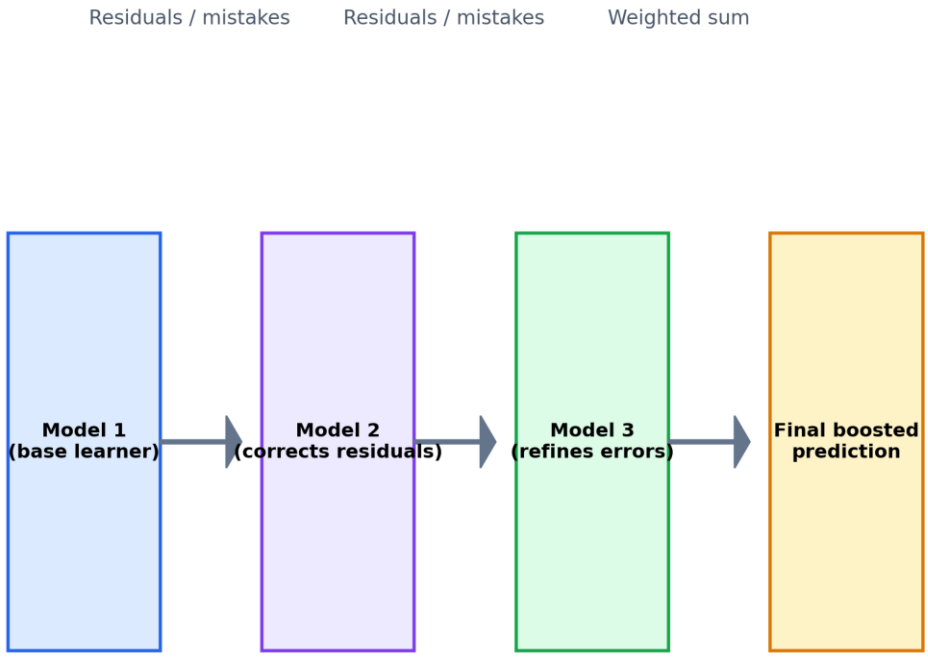
Limitations: It is less interpretable than a single decision tree and can become computationally heavier as the number of trees increases.

Real-world application: One strong example is predicting equipment failure in smart manufacturing. Sensor readings such as temperature, vibration, humidity, and pressure can be used as input features. Because industrial data is often noisy and non-linear, random forest performs well by combining many diverse trees instead of depending on one unstable rule set.

Question 4 — Gradient Boosting Classification Model

Gradient Boosting is another ensemble method, but unlike Random Forest it builds models sequentially instead of independently. It starts with a weak learner, usually a shallow decision tree, and then each new tree is trained to reduce the errors made by the previous model. In other words, every stage focuses on the remaining mistakes.

The gradient part refers to optimizing a loss function by following its gradient, while the boosting part means combining many weak learners into a stronger model. The final output is a weighted sum of the predictions from all learners.



Gradient boosting builds weak learner sequentially so that each new learner reduces the remaining error.

Strengths: Gradient boosting often achieves very high predictive performance, especially on tabular data. It can model complex non-linear relationships and handle mixed feature patterns effectively.

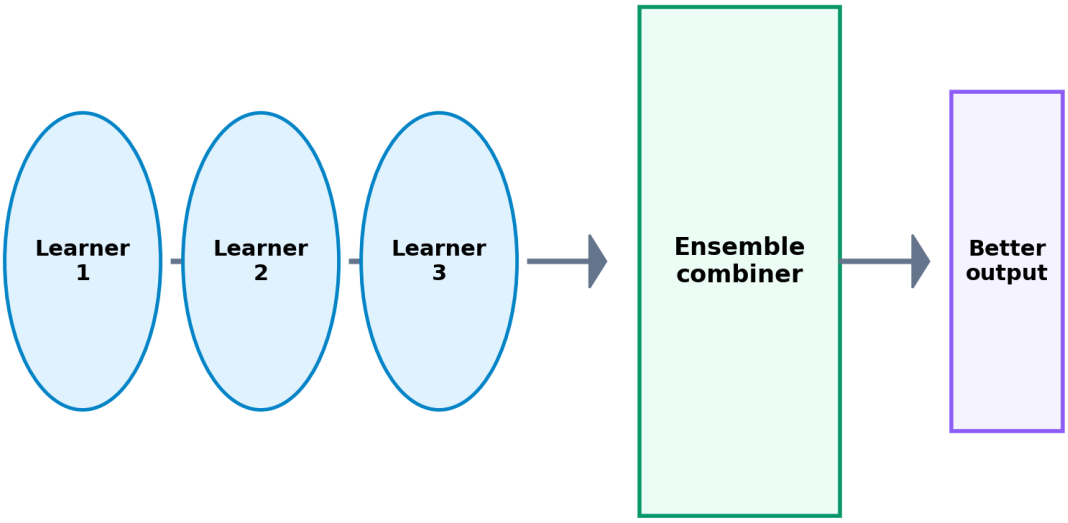
Limitations: It is more sensitive to hyperparameters and can overfit if the learning rate, tree depth, or number of estimators are not controlled carefully. Training is also slower than bagging-based methods because the learners are built one after another.

Real-world application: A practical example is employee attrition prediction. Features such as satisfaction score, overtime, department, promotion history, and salary level can be used to classify whether an employee is likely to leave. Gradient boosting is useful here because many small patterns combine to produce strong predictive power.

Question 5 — Ensemble Models

An ensemble model combines the predictions of multiple learners to produce a stronger final prediction than a single model would produce alone. The core idea is that different models make different mistakes; by combining them, the final system can reduce error and improve generalization.

The three major ensemble strategies are bagging, boosting, and stacking. Bagging mainly reduces variance, boosting mainly reduces bias, and stacking learns how to combine predictions from multiple base models through a meta-learner.



An ensemble combines multiple learner to reduce variance, bias, or both.

Example other than Random Forest: AdaBoost is a classic boosting-based ensemble model. It trains weak learners one by one and gives more attention to the samples that previous learners classified incorrectly. This makes the next learner focus on the hard cases.

Application: AdaBoost can be used in email spam detection. Many weak rules alone may not be strong enough, but when combined through boosting they can separate spam and legitimate messages more accurately.

Question 6 — Role of Activation Functions

Activation functions are mathematical functions used in neural networks to decide how strongly a neuron should respond to an input. Without activation functions, a deep neural network would behave like a simple linear model, no matter how many layers it had. Activation functions therefore introduce the non-linearity that allows deep learning models to capture complex patterns.

ReLU (Rectified Linear Unit): ReLU outputs 0 when the input is negative and outputs the input itself when it is positive. It is widely used in hidden layers of deep neural networks because it is simple, computationally efficient, and helps training converge faster.

Sigmoid: The sigmoid function maps input values to the range 0 to 1. It is commonly used in the output layer of binary classification models when the model must produce a probability.

Another common example is tanh: tanh maps values to the range -1 to 1, which can help when the data is centered around zero. It has historically been used in recurrent networks and some older architectures.

In practice, hidden layers often use ReLU or variants such as Leaky ReLU, while output layers use a task-specific activation such as sigmoid for binary classification or softmax for multi-class classification.

Question 7 — Gini Impurity

Gini impurity measures how mixed the classes are in a node of a decision tree. A node is considered pure when all samples in it belong to one class. The Gini impurity of a node is calculated as $1 - \Sigma\, pi^2$, where pi is the proportion of samples belonging to class i.

If a node contains only one class, its Gini impurity is 0. If the classes are evenly mixed, the impurity is higher. Decision tree algorithms evaluate possible splits and choose the split that reduces impurity the most.

Example: Suppose a node contains 10 samples, of which 7 are class A and 3 are class B. Then Gini impurity = $1 - (0.7^2 + 0.3^2) = 1 - (0.49 + 0.09) = 0.42$. If a candidate split produces two child nodes with lower weighted impurity, that split is considered better.

Gini impurity is popular because it is fast to compute and works well in many practical tree-based algorithms.

Question 8 — Entropy and Information Gain

In decision trees, entropy measures the amount of disorder or uncertainty in a node. When a node contains a balanced mixture of classes, entropy is high. When the node is nearly pure, entropy is low. The formula is Entropy = $-\Sigma\, pi \log2(pi)$.

Information gain tells us how much uncertainty is removed by making a split. It is calculated as the entropy of the parent node minus the weighted entropy of the child nodes. A split with higher information gain is preferred because it produces more class separation.

How these guide tree creation: At each candidate split, the algorithm evaluates how much the split reduces entropy. The split that gives the best information gain is selected. This process is repeated recursively until stopping conditions are met.

Example: If a parent node has entropy 1.0 and a candidate split creates child nodes whose weighted average entropy is 0.42, then the information gain is 0.58. That means the split removes a large amount of uncertainty and is therefore a strong candidate.

Question 9 — Sampling with Replacement

Sampling with replacement means selecting observations from a dataset in such a way that once an observation is selected, it is returned to the pool and can be selected again. Because of this, a bootstrap sample may contain repeated rows while some original rows may not appear at all.

This idea is extremely important in machine learning because it is the basis of bootstrap aggregation (bagging), which is used by algorithms such as Random Forest. Each tree in a random forest is trained on a different bootstrap sample. This diversity helps the ensemble reduce variance and generalize better.

Scenario: In a fraud detection project, a dataset of transaction records may be resampled many times to train different base learners. Each learner sees a slightly different view of the data, and the final ensemble benefits from that variation.

Question 10 — Evaluation Metrics for Classification Models

Evaluation metrics are used to judge how well a classification model performs. The best metric depends on the business objective and on the balance of the classes.

Metric	What it measures	When it is most useful
Accuracy	Overall fraction of correct predictions	Useful when classes are balanced and both error types matter similarly
Precision	Among predicted positives, how many are truly positive	Useful when false positives are expensive, such as spam flags or fraud alerts
Recall	Among actual positives, how many are correctly found	Useful when missing positives is dangerous, such as medical screening
F1-score	Harmonic mean of precision and recall	Useful when both false positives and false negatives matter
Specificity	How well the model identifies true negatives	Useful when correctly ruling out negatives matters
ROC-AUC	Ranking quality across classification thresholds	Useful for comparing classifiers over multiple thresholds
Confusion Matrix	Detailed counts of TP, TN, FP, FN	Useful for diagnosing the pattern of mistakes

For example, in cancer screening, recall is often more important than accuracy because missing a true positive case can be far more harmful than raising extra false alarms. In contrast, for marketing lead qualification, precision may matter more because the business wants to avoid spending resources on low-quality leads.

This shows that evaluation is not just a mathematical step; it must be aligned with the real-world cost of different mistakes.

Final reflection

The most important idea across these models is that classification is not only about selecting an algorithm; it is also about understanding the shape of the data, the cost of mistakes, and the reason why one model is more suitable than another.

Tree-based models are powerful because they can capture non-linear patterns, while logistic regression remains valuable because of its interpretability and stable baseline performance. Ensemble methods improve performance further by combining multiple learners instead of relying on a single model.